芯鉴知微·硬件开发指导

Oct 11, 2026 · @gusuisui

目标与范围

硬件基线已于 2026-10-11 改为同学写好并正在运行的程序：ESP32-S3-N16R8 + SHT31 温湿度传感器，数据经电脑端串口桥进入平台。原 ESP32-DevKitC + DHT11 方案作废，相关固件与资料包已从主线移除。

项目	当前状态
选型	ESP32-S3-N16R8 开发板 + Sensirion SHT31（I²C）；芯片已用 esptool 读出：ESP32-S3 rev v0.2，内置 8MB PSRAM
板上程序	同学的 ESP-IDF 程序原样入库为 `firmware/esp32s3_sht31`，登记版本 `esp32s3-sht31-1.0.0`；ESP-IDF v5.3.2 构建通过
数据通路	板子只往串口打印；电脑端串口桥 `bridge/` 解析后按设备协议 V1 上传
实验资料包	`sht31_temperature_humidity`@1.0.0（测试草稿，替换原 DHT11 包）
实物验收	**部分完成**：2026-10-11 真机 → 串口桥 → 隔离测试平台跑通，40 秒 38 行读数全部入库且数值一致；接线逐项核验、精度、故障与恢复待实测

本指导面向第一次动手的成员，给出顺序和要点；命令细节见仓库里的 `firmware/esp32s3_sht31/README.md`（板上程序）和 `bridge/README.md`（串口桥）。

硬件清单与接线

SHT31 走 I²C，只接四根线：电源、地、SDA、SCL。必须在**断电状态**下接好并拍照留证。

SHT31 引脚	接到 ESP32-S3	说明
VIN / VDD	3V3	手册供电范围 2.15–5.5 V，项目统一用 3.3 V
GND	GND	与开发板共地

SHT31 引脚	接到 ESP32-S3	说明
SDA	GPIO8	程序里 `I2C_SDA_GPIO 8`
SCL	GPIO9	程序里 `I2C_SCL_GPIO 9`
ADDR（模块上可能标 ADR）	悬空或接 GND	低电平地址 0x44（程序默认）；接 3V3 则是 0x45，需同时改程序

器材：ESP32-S3-N16R8 开发板、SHT31 模块、面包板与杜邦线、能传数据的 USB 线。目前实测用的是板子的原生 USB 口（电脑识别为 Espressif “USB JTAG/serial debug unit”，macOS 上是 `/dev/cu.usbmodem*`）。

- 程序只启用了 ESP32 内部上拉（约几十 kΩ，偏弱）。手册建议 SDA、SCL 各接外部上拉（如 10kΩ 到 3V3）；不少 SHT31 模块自带上拉，先看模块上有没有电阻再决定。
- 线尽量短（建议 20cm 以内），I²C 线长、上拉弱时容易偶发读取失败。
- N16R8 的 GPIO35、GPIO36、GPIO37 被板载 PSRAM 占用，不要用作外部引脚。

开发环境与烧录

板上程序用 ESP-IDF 构建，已验证 v5.3.2 可以编译（程序用的是旧版 `driver/i2c.h` 接口，较新的 5.x 版本可能提示弃用警告）。板子上已经烧好这个程序，只有改程序或换板时才需要重新烧录。

1. 安装 ESP-IDF v5.3.2（官方安装器或 VS Code 插件）；不想装工具链时，可用官方 Docker 镜像 `espressif/idf:v5.3.2` 编译。
2. 在 `firmware/esp32s3_sht31` 目录编译：`idf.py set-target esp32s3`，再 `idf.py build`。
3. 插拔 USB 对照确认串口后烧录：`idf.py -p <串口> flash`。
4. 串口观察（115200）：`idf.py -p <串口> monitor`，或用 COMTool 等串口助手。

串口助手里的 DTR、RTS 必须保持不勾选。ESP32-S3 的原生 USB 口把这两根控制线的特定组合当成“复位并进入下载模式”。如果串口里出现 `waiting for download`，说明板子卡在下载模式，程序没在跑；拔插一次 USB 即可恢复。

每次构建都保存日志和 `build/esp32s3_sht31.bin` 的 SHA-256，写进实测记录。

板上程序与已知限制

程序只有一个文件 `main/main.c`，按“以当前程序为主”的决定原样保留，不做修改。它每秒读一次 SHT31，把结果打印到串口。

要点	程序里的做法
I²C	`I2C_NUM_0`，SDA GPIO8、SCL GPIO9，100 kHz，内部上拉
测量	发单次测量命令 `0x2400`（高重复性、无时钟拉伸），等 20 ms（手册最长 15.5 ms）后读 6 字节
换算	温度 = −45 + 175 × 原始值 / 65535；湿度 = 100 × 原始值 / 65535
周期	每轮等 1000 ms，实测约 1.05 秒一行

串口只会出现三种行，串口桥按它们解析：

- 启动：`SHT31 传感器启动!`
- 读数：`温度: 25.95 °C | 湿度: 52.80 %RH`
- 失败：`读取失败！检查接线或者把地址改成0x45`

已知限制（记录在案，暂不修改程序）：

- **不校验 CRC**：SHT31 每个数据都带 CRC 校验字节，程序没有检查，线路干扰导致的错误数据会当作正常读数。
- **忽略发命令的返回值**：只有读数据失败（如传感器不应答）时才打印失败行。
- **没有版本号和设备时间**：版本由项目登记为 `esp32s3-sht31-1.0.0`；读数时间取电脑收到该行的时间。
- **失败原因无法区分**：接线、供电、地址、上拉、器件异常都只表现为同一句失败行。

与后端对接：串口桥与设备协议 V1

板子不联网。电脑上运行串口桥 `bridge/`，它读串口的每一行，把可识别的行换成记录，按设备协议 V1 上传到 `POST /api/v1/device/ingest`。请求头带 `X-Device-ID`、`X-Device-Token`，有实验会话时再带 `X-Experiment-Session-ID`。

串口行	上传的记录
读数行	两条 `reading`：`SHT31` 的 `temperature`（°C）和 `humidity`（%RH）
失败行	一条 `log`：级别 `error`，事件码 `SHT31_READ_FAILED`
启动行	一条 `log`：级别 `info`，事件码 `SHT31_BOOT`；同时换新的 `bootId`

串口行	上传的记录
其他行（启动日志、半行乱码）	不上传，只计数并提示

串口桥的行为：

- **成批上传**：每 5 秒或满 50 条发一批；同一次启动内 `sequenceNo` 递增。
- **心跳不补造**：只有在收到新行时才附带心跳；串口沉默时不发心跳，平台会按时判为离线。
- **时间**：`occurredAt` 是电脑收到该行的时间，记录里标注 `time_source: host_serial_receive`。
- **重试与幂等**：网络错误、408、429、5xx 会重试，重发沿用同一个 `requestId`，平台不会重复入库；平台宕机时串口照常读取，最多排队 20 批。
- **拒绝处理**：401/403 说明设备凭据不对，桥直接停止；其他 4xx 丢弃该批并计数。退出时汇报已发送、未发送、丢弃的批数。
- **测试标记**：默认 `isTestData=true`。

平台对每条记录回执 `accepted` 才算成功；`accepted` 只说明已保存，不代表学生一定看得到，记录还要落在会话的使用时间内才会挂到学生名下。单批最多 100 条、不超过 256KB、每台设备每分钟最多 120 次请求。完整字段定义见 `docs/device-protocol.md`。

联调步骤

每一步只改一个变量，出问题才好定位。2026-10-11 已在隔离环境里跑通第 1–4 步，第 5–6 步待做。

1. **启动隔离测试平台**：用一次性数据库（不要用正式业务库），迁移到最新版本，用 `python -m app.cli.create_device --device-id <设备ID> --token-env <变量名>` 建设备，再启动后端。
2. **确认板子在输出**：用 COMTool（DTR、RTS 不勾）看到每秒一行温湿度，然后**关闭 COMTool**，把串口让给串口桥（两者不能同时占用）。
3. **串口桥试跑不联网**：`python3 -m venv bridge/.venv`，`bridge/.venv/bin/pip install -e 'bridge[dev]'`，再 `bridge/.venv/bin/xinjian-serial-bridge --port <串口> --dry-run`，应看到原始行回显和生成的批次 JSON。
4. **接入平台**：用隐藏输入保存令牌（`read -rs XJ_DEVICE_TOKEN; export XJ_DEVICE_TOKEN`），运行 `xinjian-serial-bridge --port <串口> --base-url <后端地址> --device-id <设备ID>`。核对最后一行串口读数与平台入库记录一致；10-11 实测 40 秒 38 行 → 76 条读数、8 批全部接受。
5. **制造故障再恢复**：断电后只拔掉 SDA（或 SCL）再上电，串口应输出失败行，累计达到规则阈值后平台给出异常和排查步骤；恢复接线，取得新读数后在学生端点“检查当前数据”。

6. **平台中断与恢复**：运行中停掉后端约 1 分钟，串口桥应继续读行并排队；恢复后核对每个 `requestId` 只入库一次。

实物验收与常见问题

T0—T4 五道门槛都留下可追溯的原始证据，才能写“内部技术功能验收通过”。截至 2026-10-11，T0 完成，T1、T2 部分完成。

门槛	要证明什么	关键证据	状态
T0 软件就绪	程序能编译、串口桥测试通过	构建日志、固件 SHA-256、测试输出	完成（ESP-IDF v5.3.2）
T1 板上本地采集	真机能读到温湿度或如实报错	串口日志、接线照片	部分：已持续读数，缺接线照片与逐项核对
T2 平台接入	数据真实入库并归属到会话	requestId、回执、入库记录	部分：隔离环境入库一致，会话归属待做
T3 异常与恢复	拔线能触发异常，恢复后能重新判断	前后诊断 ID、证据 ID	待做
T4 平台中断与重连	后端中断不丢批次、恢复不重复入库	桥的退出汇总、幂等记录	待做

遇到问题先按现象对号：

现象	先查什么
编译失败	ESP-IDF 是否为 5.x（已验证 5.3.2）、是否先 `set-target esp32s3`
串口没输出，或出现 `waiting for download`	串口助手是否勾了 DTR/RTS、串口是否被另一个程序占用；拔插 USB 恢复
串口一直是“读取失败”	断电核对 VIN、GND、SDA→GPIO8、SCL→GPIO9，ADDR 是否拉高（0x45），上拉与线长
读数偶尔跳成离谱的值	程序不校验 CRC，先查接触和干扰，再用另一只传感器对照
串口有读数，平台没有	桥是否在运行、设备 ID 与令牌是否对应（401/403 桥会停止）、后端地址可达、退出汇总里有无丢弃
桥提示“无法识别的行”	启动日志属正常；大量乱码先查波特率 115200 和 USB 线

现象	先查什么
数据已到，诊断仍是“未知”	证据是否够触发规则（窗口内累计失败次数）、记录是否归属到会话

读取失败不能单独证明是断线、地址错还是器件损坏，平台也不会替你判定；要靠逐项对照实验排除。

后续扩展：新传感器与新实验

新增一个实验不需要改平台代码，而是做两件事：写一个新的**实验资料包**，再让设备（直接联网或经串口桥）按协议 V1 上报对应的指标。GPIO LED 实验（`gpio_led_output` `2.0.4`）就是第二个范例。

资料包放在 `backend/experiment_packages/<实验代号>/`，由 10 个数据文件组成：

文件	写什么
`metadata.yaml`	实验名称、版本、来源登记
`hardware.yaml`	器件、引脚、采样间隔、指标名与单位
`diagnosis/rules.yaml`、`fault_tree.yaml`	什么证据算异常，异常对应哪些有限候选和排查动作
`knowledge/`	概念解释与审核过的案例
`teaching/`	分级排查步骤与提示
`tests/`	正常与故障样例，发布前自动校验

约束：

- 资料包只允许数据，不能写脚本或任意表达式；规则只能用系统已注册的事实和比较运算。
- 服务端为每个文件和整个包计算 SHA-256，发布后是不可变快照；修改就升版本，历史诊断不会被改写。
- 阈值、引脚和案例先标测试草稿，经实测和教师确认后才能转为正式。
- 设备侧（固件或串口桥）保持协议 V1 不变，只新增 `metric_key` 和事件码；实测方法沿用 `docs/hardware-validation-plan.md` 的单因素试验。